addressable memory, making it a little trickier to put into use if we are not using a byte data type for our data and it requires its own library to be accessed in our code.

Now that we have an idea of what we're working with, let's count some bytes. We just mentioned how a single 32-character message when stored in an array will occupy 34 bytes of memory. When multiplied by the 20 total possible answers for our project, this single character array would consume not only 680 bytes of program memory, but when the program starts it will load this array into RAM, taking another 680 bytes of the available RAM space, as well. By the time we add in the rest of the variables and other data used by the program, we would only be left with 1043 bytes of RAM. And that's for our relatively basic example source code; as it is, by using that large of a character array, it would not have been possible to run this code on one of the Arduino Uno's predecessors that uses the ATmega168 with half as much total RAM space. Anything coded more complex with additional strings, or multiple variables, would quickly run out of RAM leading to intermittent operation and strange behavior.

Checking Free RAM

The kicker is that running out of RAM can happen at any time with absolutely no warning. When we upload our source code to the program memory on the interface board, the Arduino programming environment conveniently tells us the binary sketch size out of the maximum program memory, as in `6114 bytes (of a 32256 byte maximum)`, but we have no idea how much RAM is actually in use. Fortunately, Jean-Claude Wippler of JeeLabs wrote a small function, shown in Listing 8-3, that we can drop into a sketch to find out how much RAM is currently unused.

Listing 8-3. *freeRAM() Function Source Code*

```
int freeRAM() {
  extern int __heap_start, *__brkval;
  int v;
  return (int) &v - (__brkval == 0 ? (int) &__heap_start : (int) __brkval);
}
```

This function uses some of that ridiculous non-Arduino code that we really don't need to understand in depth, so we will just run with it. To make it work, all we need to do is drop this function into a sketch with a function call, and then open the Serial Monitor so that we can see our RAM usage. Here in Listing 8-4 is a slightly absurd example using the Blink sketch to see that even this very simple program takes 212 bytes of RAM.

Listing 8-4. *Sample freeRam() Usage*

```
int ledPin = 13;
int time = 1000;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
  Serial.println("\n[memCheck]");
  Serial.println(freeRAM(), DEC);
}
```